

!!!!

Question 2: Time-Series Forecasting – Retail Sales

Student: Saumil Mishra | SAP ID: 500121132 | Batch: 2 CCVT

!!!!

```
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.metrics import mean_absolute_error
import math

# -----
# 1. Generate Synthetic 3-Year Retail Dataset
# -----

np.random.seed(0)

dates = pd.date_range(start='2021-01-01', end='2023-12-31', freq='D')
n = len(dates)

trend = np.linspace(1000, 1800, n)
seasonal = 300 * np.sin(2 * np.pi * np.arange(n) / 365.25)
weekly = 120 * np.sin(2 * np.pi * np.arange(n) / 7)
noise = np.random.normal(0, 80, n)
sales_raw = trend + seasonal + weekly + noise
```

```

# Add 2% missing values

missing_idx = np.random.choice(n, size=int(0.02*n), replace=False)

sales_raw[missing_idx] = np.nan


df = pd.DataFrame({'Date': dates, 'Sales': sales_raw})


print("=" * 55)

print(" QUESTION 2: Time-Series Forecasting – Retail Sales")

print("=" * 55)


# -----
# 2. Preprocessing
# -----

df['Date'] = pd.to_datetime(df['Date'])

df.set_index('Date', inplace=True)

print(f"\nMissing values before: {df['Sales'].isnull().sum()}")

df['Sales'] = df['Sales'].interpolate(method='linear')

print(f"Missing values after: {df['Sales'].isnull().sum()}")

print(f"Date range: {df.index[0].date()} → {df.index[-1].date()}")

print(f"Total records: {len(df)}")


# — Plot 1: Raw Time Series —————

fig, ax = plt.subplots(figsize=(13, 4))

ax.plot(df.index, df['Sales'], color='steelblue', linewidth=0.8, alpha=0.9)

ax.set_title("Retail Store – Daily Sales (2021–2023)\n"
            "Saumil Mishra | SAP: 500121132 | Batch 2 CCVT",
            fontweight='bold')

ax.set_xlabel("Date"); ax.set_ylabel("Sales (₹)")

ax.xaxis.set_major_locator(mdates.MonthLocator(interval=3))

ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))

plt.xticks(rotation=30)

```

```

ax.grid(linestyle='--', alpha=0.4)

plt.tight_layout()

plt.savefig('/home/claude/q2_raw_series.png', dpi=150, bbox_inches='tight')

plt.close()

print("\nSaved: q2_raw_series.png")

# -----

# 3. Decomposition

# -----

result = seasonal_decompose(df['Sales'], model='additive', period=365)

fig, axes = plt.subplots(4, 1, figsize=(13, 10), sharex=True)
fig.suptitle("Time-Series Decomposition (Additive)\n"
             "Saumil Mishra | SAP: 500121132 | Batch 2 CCVT",
             fontweight='bold')

labels = ['Observed', 'Trend', 'Seasonal', 'Residual']
colors = ['steelblue', 'darkorange', 'green', 'crimson']

for ax, comp, lbl, col in zip(axes,
                              [result.observed, result.trend, result.seasonal, result.resid],
                              labels, colors):
    ax.plot(comp, color=col, linewidth=0.8)
    ax.set_ylabel(lbl, fontsize=10)
    ax.grid(linestyle='--', alpha=0.3)
    ax.xaxis.set_major_locator(mdates.MonthLocator(interval=3))
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))

plt.xticks(rotation=30)

plt.tight_layout()

plt.savefig('/home/claude/q2_decomposition.png', dpi=150, bbox_inches='tight')

plt.close()

```

```

print("Saved: q2_decomposition.png")

# -----

# 4. Stationarity Check

# -----

def adf_test(series, name="Series"):
    res = adfuller(series.dropna())
    print(f"\n— ADF Test: {name} —")
    print(f" ADF Statistic : {res[0]:.4f}")
    print(f" p-value      : {res[1]:.4f}")
    print(f" {'STATIONARY ✓' if res[1] < 0.05 else 'NON-STATIONARY X'}")
    return res[1] < 0.05

is_stat = adf_test(df['Sales'], "Original Sales")

if not is_stat:
    df['Sales_log'] = np.log(df['Sales'])
    df['Sales_diff'] = df['Sales_log'].diff().dropna()
    adf_test(df['Sales_diff'].dropna(), "Log-Differenced Sales")
    target = df['Sales_diff'].dropna()
else:
    target = df['Sales']

# — Plot 2: Differenced series —
fig, axes = plt.subplots(2, 1, figsize=(13, 6))
axes[0].plot(df['Sales'], color='steelblue', linewidth=0.7)
axes[0].set_title("Original Sales", fontweight='bold')
axes[0].grid(linestyle='--', alpha=0.4)
axes[1].plot(df['Sales_log'].diff().dropna(), color='darkorange', linewidth=0.7)
axes[1].set_title("Log-Differenced Sales (Stationary)", fontweight='bold')
axes[1].grid(linestyle='--', alpha=0.4)

```

```
fig.suptitle("Stationarity Transformation\nSaumil Mishra | SAP: 500121132 | Batch 2 CCVT",
            fontweight='bold')
plt.tight_layout()
plt.savefig('/home/claude/q2_stationarity.png', dpi=150, bbox_inches='tight')
plt.close()
print("Saved: q2_stationarity.png")
```

```
# _____
```

```
# 5. SARIMA Model
```

```
# _____
```

```
print("\n— Fitting SARIMA(1,1,1)(1,1,1,7) —")
```

```
train = df['Sales'][:-30]
```

```
test = df['Sales'][-30:]
```

```
model = SARIMAX(train,
                 order=(1,1,1),
                 seasonal_order=(1,1,1,7),
                 enforce_stationarity=False,
                 enforce_invertibility=False)
```

```
fitted = model.fit(dispatch=False)
```

```
print(fitted.summary().tables[0])
```

```
# _____
```

```
# 6. Forecast Next 30 Days
```

```
# _____
```

```
forecast_obj = fitted.get_forecast(steps=30)
```

```
forecast = forecast_obj.predicted_mean
```

```
conf_int = forecast_obj.conf_int()
```

```
mae = mean_absolute_error(test, forecast)
```

```
rmse = math.sqrt(np.mean((test.values - forecast.values)**2))
```

```

print(f"\n— Evaluation —")
print(f" MAE : {mae:.2f}")
print(f" RMSE : {rmse:.2f}")

# — Plot 3: Forecast —
fig, ax = plt.subplots(figsize=(13, 5))
ax.plot(train[-90:], color='steelblue', label='Training (last 90 days)', linewidth=1)
ax.plot(test, color='darkorange', label='Actual (test)', linewidth=1.5)
ax.plot(forecast, color='green', label='Forecast', linewidth=1.5, linestyle='--')
ax.fill_between(conf_int.index,
                conf_int.iloc[:,0], conf_int.iloc[:,1],
                color='lightgreen', alpha=0.35, label='95% CI')
ax.set_title(f"SARIMA Forecast – Next 30 Days\n"
             f"MAE={mae:.1f} RMSE={rmse:.1f} | "
             f"Saumil Mishra | SAP: 500121132 | Batch 2 CCVT",
             fontweight='bold')
ax.set_xlabel("Date"); ax.set_ylabel("Sales (₹)")
ax.legend(); ax.grid(linestyle='--', alpha=0.4)
ax.xaxis.set_major_locator(mdates.WeekdayLocator(interval=2))
ax.xaxis.set_major_formatter(mdates.DateFormatter('%d %b'))
plt.xticks(rotation=30)
plt.tight_layout()
plt.savefig('/home/claude/q2_forecast.png', dpi=150, bbox_inches='tight')
plt.close()
print("Saved: q2_forecast.png")

```

```

# —
# 7. Monthly Aggregation – Trend View
# —
monthly = df['Sales'].resample('ME').mean()
fig, ax = plt.subplots(figsize=(11, 4))

```

```
ax.bar(monthly.index, monthly.values, width=20, color='steelblue', alpha=0.8)
ax.plot(monthly.index, monthly.values, 'o-', color='darkorange', linewidth=1.5)
ax.set_title("Monthly Average Sales Trend\n"
             "Saumil Mishra | SAP: 500121132 | Batch 2 CCVT",
             fontweight='bold')
ax.set_xlabel("Month"); ax.set_ylabel("Avg Daily Sales (₹)")
ax.xaxis.set_major_locator(mdates.MonthLocator(interval=2))
ax.xaxis.set_major_formatter(mdates.DateFormatter('%b %Y'))
plt.xticks(rotation=30)
ax.grid(axis='y', linestyle='--', alpha=0.4)
plt.tight_layout()
plt.savefig('/home/claude/q2_monthly_trend.png', dpi=150, bbox_inches='tight')
plt.close()
print("Saved: q2_monthly_trend.png")

print("\n✓ All Q2 plots generated successfully.")
```